

PHASE 7 – TESTING, VALIDATION & CONCLUSION

1. Testing & Validation

The implemented UI Policies and Client Scripts must be tested to ensure that all configured conditions, validations, and form behaviors work as expected.

2. Testing 1 – Mandatory Enforcement

Test Steps

1. Navigate to **Incident** → **Create New**.
2. Set **Impact = High**.
3. Leave the **Assigned To** field empty.
4. Click **Submit**.
5. Verify that the Incident record is not saved.
6. Verify that the appropriate error message is displayed.

Expected Result

The Incident record should **not be saved** because the **Assigned To** field is required for a High Impact Incident.

3. Testing 2 – Successful Save

Test Steps

1. Set **Impact = High**.
2. Enter a valid value in the **Assigned To** field.

3. Click **Submit**.
4. Verify that the Incident record is saved successfully.

Expected Result

The Incident record should be saved successfully without validation errors.

4. Testing 3 – Reverse Condition

Test Steps

1. Open an Incident where **Impact = High**.
2. Change **Impact** from **High** to **Medium**.
3. Verify that the **Assignment group** field is no longer mandatory.
4. Verify that the **Urgency** field becomes editable.
5. Save the Incident record.

Expected Result

The UI Policy changes should be reversed when the configured condition is no longer true.

The Assignment group should no longer be mandatory, and the Urgency field should become editable according to the configured UI Policy Actions.

5. Testing 4 – List Edit Blocking

Test Steps

1. Navigate to **Incident** → **All**.
2. Select an Incident record.
3. Try to edit the **State** field directly from the list.
4. Verify that the alert message is displayed.

Expected Result

The State update should be blocked by the **onCellEdit Client Script**.

The user should be instructed to open the Incident form to make the State change.

6. Testing 5 – Form-Based Update

Test Steps

1. Open an Incident record.
2. Change the **State** field from the Incident form.
3. Click **Update**.
4. Verify that the State change is saved successfully.

Expected Result

State changes made through the Incident form should be saved successfully, subject to any other applicable validation or access controls.

7. Test Summary

Test Case	Scenario	Expected Result	Status
TC01	High Impact with Assigned To empty	Record should not save	Pass
TC02	High Impact with Assigned To populated	Record should save	Pass
TC03	Change Impact from High to Medium	UI Policy behavior should reverse	Pass
TC04	Change State using list edit	Update should be blocked	Pass
TC05	Change State from Incident form	State change should save	Pass

8. Final Outcome

The project successfully demonstrates the use of **ServiceNow UI Policies and Client Scripts** to control Incident form behavior and improve data validation.

The implementation provides the following capabilities:

- Conditional mandatory fields.
- Automatic urgency assignment.
- Read-only field control.
- Save-time validation.
- List-edit restrictions.
- Dynamic reversal of UI behavior.
- Improved Incident data consistency.
- Better control over Incident form interactions.

9. Conclusion

The **Implement Client Script & UI Policy (Incident)** project demonstrates how client-side controls can improve data quality, consistency, and user interaction in ServiceNow.

UI Policies dynamically control field behavior based on defined conditions, while **onChange, onSubmit, and onCellEdit Client Scripts** provide automation, validation, and control over user actions.

The completed solution helps ensure that Incident records contain the required information before submission and reduces inappropriate or unintended list-based modifications.

Overall, the implementation provides a structured approach to improving Incident data integrity and enforcing business requirements within the ServiceNow Incident Management module.